0. **Code:** Sirasanambeti Himaja.zip

1. Sirasanambeti Himaja / A / A / Program.cs
2. Sirasanambeti Himaja / B / Program.cs
3. Sirasanambeti Himaja / C / Program.cs

1. **Sirasanambeti Himaja / A / A / Program.cs**

```
using System.Diagnostics.CodeAnalysis;

namespace A
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter an Integer");
            int number = Convert.ToInt32(Console.ReadLine());


            int BaseN = 0;


            while (BaseN ≤ 0)
            {

                Console.WriteLine("Enter an Base-N");
                BaseN = Convert.ToInt32(Console.ReadLine());


            }

            int sum = 0;
            int maxdigit = 0;
            int value = Math.Abs(number);


            while (value > 0)
            {
```

A1

A2

A5

A3

1. **Sirasanambeti Himaja / A / A / Program.cs**

```
                int digit = value % BaseN;
                sum += digit;
                if (digit > maxdigit)
                {
                    maxdigit = digit;
                }
                value = value / BaseN;
            }
            Console.WriteLine($"sum of Base-{BaseN} digits:{sum}.");

            Console.WriteLine($"max base-{BaseN} digit:{maxdigit}.");



        }



    }



}
```

2. Sirasanambeti Himaja / B / Program.cs

```
using System.Reflection.Metadata.Ecma335;

namespace B
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] result = {
                23,
                1,
                10,
                5,
                2
            };
            int[] counts = insertioncounts(result);
            Console.WriteLine(counts);
            for (int i = 0; i < counts.Length; i++)
            {
                Console.WriteLine(counts[i]);
            }
        }
        static int[] insertioncounts(int[] array)
        {
            int[] newArray = new int[array.Length];
            for (int i = 0; i < array.Length; i++)
            {
                int value = 0;
                for (int j = 0; j < i; j++)
                {

```

B1

B2

B3

good

2. **Sirasanambeti Himaja / B / Program.cs**

```
                    if (array[j] > array[i])
                    {
                        value++;
                    }

                }
                newArray[i] = value++;
            }
            return newArray;
        }

    }
}
```

B3

Value i

0/c

B4 Without Sorting.

3. **Sirasanambeti Himaja / C / Program.cs**

```
using System.Diagnostics.CodeAnalysis;
using System.Reflection.Metadata.Ecma335;

namespace C
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] inputarray = {
                1,
                2,
                3,
                2,
                7,
                3,
                4,
                4,
                5,
                5,
                4,
                5,
                1,
                6,
                1,
                1
            };
            int[] result = largestnumbersrepeated(inputarray);
            Int result =
            Console.WriteLine("result");

        }
```

3. Sirasanambeti Himaja / C / Program.cs

```
        static int[] largestnumbersrepeated(int[] array)
        {
            if (array == null || array.Length == 0)
            {
                return null;
            }
            Dictionary < int, int > countingoccurrences = new Dictionary
              < int, int > ();
            Dictionary < int, int > differentfrequencies = new Dictionary
              < int, int > ();
            foreach(int i in array)
            {
                if (countingoccurrences.ContainsKey(i))
                {
                    countingoccurrences[i]++;
                }
                else
                {
                    countingoccurrences[i] = 1;
                }
            }
            foreach(var num in countingoccurrences)
            {
                int number = num.Key;
                int frequency = num.Value;
                if (differentfrequencies.ContainsKey(frequency) || number >
                  differentfrequencies[frequency])
                {
                    differentfrequencies[frequency] = number;
                }

```

int

C5

C2

C3

3. **Sirasanambeti Himaja / C / Program.cs**

```
            }
            int sum = 0;
            foreach(var num in differentfrequencies)
            {
                sum += num.Key * num.Value;
            }
            return array;
        }
    }
}
```

sum;

C2 okay

C1 Return the sum, not array.

C6 good!